

JARVIS CRM — DEEP PRODUCTION-LEVEL AUDIT REPORT

Audit mode: Analysis only

Fixing/debugging: NOT performed

Primary principle: Trace Database → Model → Service → API → Frontend → Persistence before changing anything.

1. EXECUTIVE SUMMARY

JARVIS CRM is architecturally ambitious and already contains a substantial amount of functionality: multi-tenant organizations, Super Admin controls, ORG_ADMIN workflows, Telecaller Desk, leads, pipelines, assignments, tasks, follow-ups, imports, Global Intelligence, Products & Services, masking, audit logging, Celery/Redis infrastructure, billing, integrations and frontend role-based navigation.

However, the current implementation is not yet internally consistent enough to be treated as a production-safe multi-tenant CRM.

The most important architectural problem is that several features were implemented incrementally using different representations of the same business concept.

Examples include:

- Telecaller targets exist both as `users.telecaller_targets` JSON and `telecaller_targets` rows.
- Follow-ups are generated by the backend policy engine while the frontend independently creates follow-up tasks.

- Calls can create both a `CallRecord` and one or more `Activity` records, while reporting counts Activities as calls.
- Lead stage and lead status can both represent lifecycle state, but the generic lead update endpoint can directly modify status without going through stage transition logic.
- Global Registry pull state exists partly in global tables, partly in tenant CRM records, and historically in `global_data_pull_logs`, but the current pull service does not establish a single atomic claim ledger.
- Tenant isolation is implemented using both `organization_id` filtering and PostgreSQL schema-per-tenant behavior, while some tenant-owned tables remain in `public`.
- Alembic migrations, SQL schema files, bundled SQLite databases and SQLAlchemy models are not synchronized.

Therefore, the application currently has a **functional feature set but an unstable source-of-truth architecture**.

2. OVERALL HEALTH ASSESSMENT

Overall status

Architecture: PARTIALLY SOUND / DRIFTING

Database integrity: HIGH RISK

Backend/API correctness: HIGH RISK

Frontend functionality: MODERATE RISK

Authentication: GOOD FOUNDATION / IMPORTANT GAPS

Authorization: HIGH RISK

Tenant isolation: HIGH RISK

Telecaller workflow: HIGH RISK

Follow-up engine: HIGH RISK

Target engine: HIGH RISK

Global Registry: CRITICAL RISK

Import system: HIGH RISK

Products & Services: MODERATE/HIGH RISK

Attendance/Shift: MODERATE/HIGH RISK

Integrations: HIGH RISK

Testing: INSUFFICIENT FOR CURRENT RISK LEVEL

Migration readiness: CRITICAL RISK

Production readiness: NOT READY without remediation of the Critical/High findings.

3. SOURCE-OF-TRUTH ARCHITECTURE

The supplied project Bible defines service-layer dominance and states that business logic should reside exclusively in services, with tenant identity derived server-side and tenant-owned rows carrying `organization_id`.

The actual implementation does not consistently follow that architecture.

For example:

`backend/app/api/v1/leads.py`

contains substantial business logic directly inside API endpoints.

`backend/app/api/v1/telephony.py`

creates `CallRecord` and `Activity` objects directly.

`backend/app/api/v1/payments.py`

directly changes Lead status to `WON`.

`frontend/src/components/TelecallerDesk.tsx`

directly decides when a follow-up task should be created.

This creates a situation where the same business rule can be implemented differently in multiple layers.

The application therefore needs a formal source-of-truth matrix before implementation work begins.

4. CRITICAL FINDINGS

C-01 — Migration chain is not a reliable representation of the current database

Severity: CRITICAL

The Alembic chain contains destructive and contradictory changes.

`a7a34b235040_add_product_service_and_import_logic.py`

drops:

- `global_edit_requests`
- `global_companies.pulled_at`
- `global_companies.pulled_by_org_id`
- `global_companies.pull_status`
- `global_companies.pulled_by_org_name`
- equivalent `global_people` columns.

Then:

`651077a665a1_batch_updates_phase10_13.py`

attempts to create indexes on:

`global_companies.pulled_by_org_id`

and

`global_people.pulled_by_org_id`.

Those columns were already removed by the previous migration.

The same migration also drops `global_edit_requests`, while the application still contains:

`backend/app/models/data_governance.py`

and:

`backend/app/api/v1/global_edits.py`

using global edit request functionality.

This means the application model layer and migration head do not represent the same schema.

Impact

A clean production database cannot be trusted to reach the schema represented by the current application.

5. C-02 — Startup

`Base.metadata.create_all()` hides migration failures

Severity: CRITICAL

`backend/app/main.py`

contains:

```
Base.metadata.create_all(bind=engine)
```

during application startup.

This is dangerous in production because the application can create tables that Alembic never successfully created.

The startup code also performs additional manual `ALTER TABLE` operations.

This creates three competing schema mechanisms:

1. Alembic
2. SQLAlchemy `create_all`
3. manual startup schema mutation

Therefore, a developer can see a working application and incorrectly conclude that migrations are correct when the application is actually relying on startup-created tables.

This is especially serious because the clean migration test already failed.

6. C-03 — Clean migration verification fails

Severity: CRITICAL

I attempted:

```
alembic upgrade head
```

against a fresh SQLite database.

The chain reached:

```
3280ba601d18_catchup_missing_columns
```

and failed at:

```
batch_op.create_foreign_key(None, ...)
```

because SQLite/Alembic requires a named constraint for that operation.

Therefore the documented claim that the migration system supports SQLite seamlessly is not currently true.

More importantly, this exposes that migrations have not been validated as a complete clean-database build.

7. C-04 — Hardcoded production fallback JWT secret

Severity: CRITICAL

`backend/app/core/config.py` contains a default:

```
jarvis-crm-super-secret-key-production-change-me-32-chars-minimum-xyz
```

If `SECRET_KEY` is not correctly configured, the application can run using a predictable source-controlled secret.

For a real CRM containing customer PII, this is unacceptable.

The configuration also defaults to:

```
DEBUG=True
```

which increases production risk.

8. C-05 — Global exception handler leaks internal errors

Severity: CRITICAL

```
backend/app/main.py
```

has a global exception handler that returns:

```
str(exc)
```

to the client.

Therefore database errors, implementation errors, internal paths, validation errors and potentially sensitive operational information can be exposed directly through API responses.

The handler also manually constructs:

```
Access-Control-Allow-Origin: <request origin>
```

and:

```
Access-Control-Allow-Credentials: true
```

which is unsafe when combined with unrestricted origins.

9. C-06 — Telecaller same-tenant IDOR

Severity: CRITICAL

The lead listing endpoint correctly restricts Telecallers to assigned leads.

However, the individual lead endpoint:

```
GET /leads/{id}
```

only checks:

```
Lead.id == id
```

and:

```
Lead.organization_id == tenant_id
```

It does not apply the Telecaller ownership/delegation rule used by the listing endpoint.

Therefore:

Telecaller → manipulate lead ID → retrieve another Telecaller's lead

is possible within the same organization.

The same issue affects other lead-specific endpoints such as:

- `/leads/{id}/pre-call-context`
- `/leads/{id}/timeline`
- potentially `/leads/{id}/stage-history`
- `/leads/{id}/action/{action_type}`
- generic lead update operations.

The frontend hiding a lead does not solve this.

10. C-07 — Telecaller can modify leads without ownership enforcement

Severity: CRITICAL

`PATCH /leads/{id}` checks tenant ownership but does not enforce Telecaller ownership.

Therefore a Telecaller may potentially update another Telecaller's lead if they know its ID.

This violates the intended rule:

| A Telecaller may operate only on leads explicitly authorized for that Telecaller.

The authorization must be enforced at the backend service/domain level, not only in list endpoints.

11. C-08 — ORG_ADMIN permission override can become privilege escalation

Severity: CRITICAL

`backend/app/models/user.py`

shows that `permission_overrides` are included in effective platform permissions.

However:

`backend/app/api/v1/users.py`

allows ORG_ADMIN to update:

`permission_overrides`

for users within the organization.

That means an ORG_ADMIN potentially controls a field that `has_platform_permission()` uses for platform-level authorization.

This creates a possible privilege escalation path:

ORG_ADMIN → modify `permission_overrides` → grant platform permission → access protected platform functionality.

This needs to be treated as a major security boundary issue.

12. C-09 — Telephony webhook has no authentication/signature validation

Severity: CRITICAL

`POST /telephony/webhook`

does not use:

`get_current_user`

or any provider signature verification.

It accepts:

- call ID
- status
- disposition
- duration
- recording URL

and modifies a CallRecord.

It also creates a CRM Activity.

Therefore an external party who can reach the endpoint could potentially fabricate call events.

This directly affects:

- productivity
 - targets
 - call history
 - talk time
 - reporting
 - potentially customer-visible records.
-

13. C-10 — Payment webhook has no provider signature verification

Severity: CRITICAL

POST /payments/webhook

is intentionally public, but there is no visible provider signature verification.

The endpoint can locate a payment and mark it:

PAID

and then transition the associated Lead to:

WON .

This is a direct business-state transition.

A forged webhook could therefore produce:

Payment → PAID → Lead WON.

This must not be accepted without authenticated provider verification.

14. C-11 — Any authenticated tenant user can simulate payment

Severity: CRITICAL

POST /payments/{payment_id}/simulate-payment

uses tenant context but does not enforce ORG_ADMIN or another privileged role.

It changes:

```
payment.status = "PAID"
```

and:

```
lead.status = "WON".
```

This is not merely a demo/UI issue because it directly mutates a business-critical lifecycle state.

15. C-12 — Lead status can bypass pipeline lifecycle

Severity: CRITICAL

The application correctly has:

```
change_lead_stage()
```

for stage transitions and stage history.

However `PATCH /leads/{id}` directly accepts:

```
data.status
```

and assigns:

```
lead.status = data.status.upper()
```

without invoking the stage transition service.

Therefore a lead can potentially have:

- status = WON
- pipeline stage = Proposal

or:

- status = LOST
- pipeline stage = Contacted

without corresponding stage history.

This directly violates the requirement that lead state and conversion state must have a single authoritative lifecycle.

16. C-13 — Product/Service foreign-key validation is incomplete on lead update

Severity: CRITICAL

Lead creation correctly validates:

```
ProductService.id == requested ID
```

and:

```
ProductService.organization_id == organization_id.
```

However the generic lead update endpoint directly assigns:

```
lead.product_service_id = data.product_service_id
```

without the same organization-level validation.

This means the create and update flows have different security rules.

A cross-tenant Product/Service reference may therefore be introduced through update operations.

17. C-14 — Global Registry pull quota is deducted for requested records, not successful claims

Severity: CRITICAL

```
global_service.py
```

does:

```
needed = len(global_company_ids)
```

and eventually:

```
sub.pull_quota_used += needed
```

even though individual records can be:

- missing
- already present
- not newly created.

This exactly creates the previously identified quota bug.

Quota must represent **successful claims**, not requested IDs.

18. C-15 — Global Registry claiming is not atomic

Severity: CRITICAL

The pull operation:

- reads a global company
- checks whether a tenant Company exists
- creates it if absent
- creates a Lead
- commits later.

There is no `SELECT ... FOR UPDATE` /row locking around the global record.

There is also no reliable unique claim ledger used by the operation.

Two organizations can concurrently pull the same global record.

The current design therefore does not provide atomic global claiming.

19. C-16 — Global pull lineage is incomplete

Severity: CRITICAL

The Bible describes:

`global_data_pull_logs`

as the lineage mechanism.

But the current model does not contain a corresponding SQLAlchemy model, and the current pull service does not create a pull-log record.

The application instead determines whether a company was pulled by querying tenant `Company.source_global_company_id`.

That is not equivalent to a centralized claim ledger.

Consequently:

- pull history is incomplete
 - claim ownership is not authoritative
 - repeated claims are not centrally controlled
 - audit lineage is weaker than documented.
-

20. C-17 — Global pull target owner is not validated

Severity: CRITICAL

Global pull accepts:

`target_owner_id`

from the client.

The service uses it directly:

`owner_id = target_owner_id or user.id`

without sufficiently proving that the target user:

- belongs to the target organization
- is active
- is a valid Telecaller
- is permitted to receive the lead.

This creates a potential cross-tenant ownership contamination path.

21. C-18 — Global Registry exposes raw PII to insufficiently restricted roles

Severity: CRITICAL

`/global/companies`

and `/global/people`

are authenticated but do not consistently enforce the same role restrictions used elsewhere.

Their response models include:

- email
- phone
- LinkedIn
- address
- other contact information.

The Telecaller frontend hides Global Registry navigation, but the API cannot rely on frontend hiding.

A Telecaller who directly calls the endpoint may potentially retrieve raw global PII.

This bypasses the application's stated masking model.

22. C-19 — Tenant schema names are not guaranteed unique

Severity: CRITICAL

`get_schema_name_for_org()` derives PostgreSQL schema names from organization names and truncates them to 60 characters.

`organizations.schema_name` is indexed but not shown as unique.

Two organizations with similar long names can therefore potentially resolve to the same schema.

For a schema-per-tenant architecture, this is a serious isolation risk.

23. C-20 — Telecaller call metrics are structurally inconsistent

Severity: CRITICAL

```
POST /telephony/dial
```

creates:

1. `CallRecord`
2. `Activity(activity_type="CALL")`

Then the webhook creates another:

1. `Activity(activity_type="CALL")`

Then the Telecaller Desk separately calls:

```
POST /activities
```

when the user records an outcome.

Therefore one real phone call can produce multiple CALL Activities.

Yet target and KPI logic counts Activity rows as calls.

This can cause:

1 real call → 2 or 3 counted calls

depending on execution path.

This directly corrupts target achievement and productivity reporting.

24. C-21 — Telecaller frontend uses fabricated call duration

Severity: CRITICAL

```
TelecallerDesk.tsx
```

records:

```
duration_seconds: outcome === "CONNECTED" ? 120 : 15
```

This means the frontend creates business telemetry using hardcoded durations. The actual call duration is not being sourced from the telephony provider/device. Therefore talk time can be inaccurate by design.

25. C-22 — Target source of truth is duplicated

Severity: CRITICAL

There are two target representations.

Representation A

`users.telecaller_targets`

JSON:

```
{
  calls,
  connects,
  conversions
}
```

ORG_ADMIN edits this through:

`PATCH /users/{id} .`

Representation B

`telecaller_targets`

contains:

- target_calls
- target_connects
- target_talk_time_minutes
- target_qualified_leads
- target_conversions

- target_revenue
- target_date.

The daily target engine reads Representation B.

The Team Management UI writes Representation A.

Therefore:

ORG_ADMIN writes one source. Telecaller Desk reads another source.

This is a fundamental source-of-truth failure.

26. C-23 — Target engine contains arbitrary hardcoded defaults

Severity: CRITICAL

`daily_tasks.py` creates:

- Calls = 80
- Connects = 30
- Talk time = 120
- Qualified leads = 10
- Conversions = 2

EOD logic also falls back to:

```
target_calls = target.target_calls if target else 80
```

These are not explicitly documented organization-level defaults.

This violates the requirement that targets come from authoritative configuration rather than silently substituting arbitrary values.

27. C-24 — Target dates use UTC instead of organization timezone

Severity: CRITICAL

The daily target engine uses:

```
datetime.now(timezone.utc).date()
```

The organization model already contains a timezone.

The same issue appears in EOD calculations.

For an Indian organization, midnight UTC occurs at approximately 5:30 AM IST.

Therefore a "business day" is not necessarily aligned with the organization's actual day.

This can corrupt:

- daily targets
- calls
- conversions
- EOD reports
- shift data.

28. C-25 — EOD report schedule does not represent actual EOD

Severity: CRITICAL

Celery is configured with:

```
timezone="UTC"
```

and:

```
generate-eod-reports at:
```

```
23:30 UTC .
```

For India this is approximately:

05:00 AM IST next day.

The code comment describes an EOD process, but execution is based on UTC rather than organization timezone.

This is a clear scheduling/business-time mismatch.

29. HIGH-SEVERITY FINDINGS

H-01 — Follow-up engine and frontend both create follow-ups

`activity_service.py` automatically invokes:

```
execute_followup_policy()
```

for CALL activities.

Then `TelecallerDesk.tsx` explicitly calls:

```
api.createTask(...)
```

for the same call outcome.

Therefore the follow-up engine and frontend both participate in task creation.

The task service may cancel/recreate tasks, but this does not solve the architectural problem.

There should be one authoritative follow-up creation mechanism.

H-02 — Follow-up policy can be overridden by frontend presets

The frontend chooses:

- tomorrow
- 3 days
- next week.

The backend policy engine independently decides whether and when a follow-up should exist.

The two mechanisms can disagree.

Therefore follow-up policy is not truly authoritative.

H-03 — No database-level uniqueness for active follow-ups

The task model does not appear to enforce a unique constraint preventing: same organization + lead + active FOLLOW_UP from existing multiple times.

Application-level idempotency alone is insufficient under concurrent requests.

H-04 — Rescheduling history is inconsistent

`followups.py` directly changes:

`task.due_at`

and appends text to `task.notes`.

The main task service contains separate rescheduling logic and `reschedule_history`. Therefore there are multiple rescheduling implementations.

H-05 — Activity API allows same-tenant Telecaller activity against arbitrary leads

`activity_service.py` verifies organization but not Telecaller ownership.

Therefore the same same-tenant IDOR problem exists in activity creation.

H-06 — `get_activities()` accepts arbitrary user_id

The API allows a `user_id` filter.

Although the tenant is filtered, Telecaller access does not appear to force `user_id = current_user.id`.

This should be checked as a potential productivity/history information leak.

H-07 — Generic Lead Update has inconsistent field semantics

`LeadUpdate` contains multiple fields, but the endpoint selectively applies only some.

For example ownership-related fields may exist in the schema but are not treated consistently.

This creates frontend/backend contract ambiguity.

H-08 — Assignment has both `owner_id` and `lead_assignments`

The lead has:

`owner_id`

while the database also has:

`lead_assignments`.

The system therefore maintains ownership in two places.

The application must explicitly define:

`owner_id` = current primary assignment

and guarantee synchronization.

Otherwise these records can diverge.

H-09 — Assignment history is not protected by database uniqueness

The application can create assignment records, but there is no strong database-level invariant guaranteeing exactly one active primary assignment.

Concurrency can therefore create inconsistent ownership state.

H-10 — Global People and Global Contacts represent overlapping concepts

The project contains:

`global_contacts`

and:

`global_people`.

It also has:

`GlobalContact`

and:

`GlobalPerson` .

The Bible describes `global_contacts` as the global people registry, while current implementation separately uses `global_people` .

This is another source-of-truth duplication.

H-11 — Duplicate function definitions exist

`global_service.py` defines:

`update_global_company`

twice.

The later definition silently replaces the earlier definition.

`global_people_service.py` defines:

`update_global_person`

twice.

This is a code-quality and maintenance defect and can cause developers to edit the wrong implementation.

H-12 — Tenant schema architecture is hybrid

The application uses:

- `organization_id`
- public tables
- PostgreSQL tenant schemas
- `search_path`
- startup schema cloning
- fallback to `public` .

This is more complex than a simple shared-schema architecture.

The tenant table list includes 34 tables, while many application models remain public.

Some clearly tenant-owned models such as Products & Services and API management are not part of the tenant schema list.

Therefore the application currently has a hybrid isolation model.

H-13 — Tenant schema synchronization uses manual SQL instead of migrations

`tenant_schema.py` creates tables using:

```
CREATE TABLE ... LIKE public.table INCLUDING ...
```

and then separately applies manual column additions.

This means schema evolution can occur independently of Alembic.

That creates another schema source of truth.

H-14 — Tenant schema clone does not guarantee complete relational integrity

The schema cloning approach focuses on copying table structures and indexes.

The application should explicitly verify that foreign keys and relational constraints are actually reproduced correctly inside tenant schemas.

Otherwise tenant-local tables may exist without the intended relational integrity.

H-15 — Startup migration errors are swallowed

Several schema operations use:

```
except Exception: pass
```

or logging-only behavior.

This means a failed schema mutation can leave the application running against an incomplete database.

For a production CRM, schema failures should fail startup rather than silently continue.

H-16 — `Session.refresh()` globally monkey-patched to swallow every exception

`database.py` replaces:

`Session.refresh`

and catches:

`Exception`.

This is extremely broad.

A real database problem can therefore become a silently ignored refresh problem.

This makes debugging much harder and can hide persistence inconsistencies.

H-17 — Import accepts client-supplied server file paths

`ImportJobExecuteRequest` contains:

`file_path`.

The execute endpoint accepts this path and checks whether it exists.

This creates unnecessary path-trust risk.

The server should issue an opaque upload/job ID and resolve the physical path internally.

H-18 — Import upload has no effective file-size enforcement

The configuration contains:

`MAX_UPLOAD_SIZE_MB = 50`

but the upload endpoint reads the entire file:

`content = await file.read()`

without visibly enforcing the configured limit.

This can cause memory exhaustion for large uploads.

H-19 — Import file types are broader than the actual business requirement

The system allows:

- CSV
- XLS
- XLSX
- XLSB
- XLSM
- PARQUET
- JSON.

Each format has different parsing/security characteristics.

The application needs explicit parser-level validation and limits rather than extension-only validation.

H-20 — Imported Product/Service validation must be enforced server-side

The schema supports:

```
default_product_service_id.
```

The final import execution path must guarantee that the product belongs to the current organization.

This needs dedicated regression coverage.

H-21 — Payment lead lookup is inconsistently tenant scoped

The payment webhook locates a Payment globally and then loads:

```
Lead.id == payment.lead_id
```

without always carrying the payment organization into the lead query.

Because the payment itself was globally resolved, the code relies on relational correctness rather than consistently enforcing tenant scope at every query boundary.

H-22 — Payment lifecycle lacks idempotency

Repeated provider webhooks can repeatedly execute payment state transitions and Activity creation.

There is no obvious idempotency key controlling webhook replay.

H-23 — Communication layer contains simulated SMS/WhatsApp dispatch

The templated communication dispatcher has:

```
elif channel in ("WHATSAPP", "SMS"): pass
```

yet still writes:

```
status="SENT" .
```

This means the database can report a message as SENT when no actual provider dispatch occurred.

That is a major integration/source-of-truth mismatch.

H-24 — WhatsApp direct endpoint also simulates successful delivery

The direct WhatsApp endpoint creates a communication record with:

```
status="SENT"
```

without demonstrating actual provider delivery.

This should not be treated as a successful external communication event.

H-25 — Frontend email behavior is partly copy-only

The current frontend copies email addresses in some workflows rather than providing a complete CRM email lifecycle.

This is not inherently a security bug, but it means UI functionality can appear more complete than the underlying integration.

H-26 — Products & Services frontend contract mismatch

Frontend defines:

```
status
```

while backend ProductService uses:

```
is_active .
```

The frontend delete action sends:

```
{ is_active: false }
```

but the frontend interface still expects:

```
status .
```

This means UI state can become inconsistent with backend state.

H-27 — Products & Services is tenant-owned but not included in tenant schema list

`product_services` is absent from `TENANT_TABLES` .

Therefore it relies on the public schema.

This may work because queries use organization filters, but it violates the stronger schema-isolation model documented elsewhere.

H-28 — Global company updates have no organization-independent audit mechanism

Global records are platform-level master records, but the update service does not appear to consistently record a formal global change history equivalent to the intended governed change-request mechanism.

H-29 — Global Registry master record mutation rules are inconsistent

The application documentation describes global master data as controlled/read-only for tenants.

Yet create/update permissions and service implementations have different role checks and duplicated update functions.

The authoritative mutation policy needs to be centralized.

H-30 — Global Registry PII response models are not role-specific

`GlobalCompanyOut` and related models always contain raw PII fields.

A role-safe response DTO would be safer than relying entirely on frontend behavior.

30. MEDIUM-SEVERITY FINDINGS

M-01 — Pagination defaults are high

Lead listing defaults to:

`limit=250`.

Other APIs use different limits.

There is no common pagination standard.

M-02 — Pagination lacks total-count metadata

Most list endpoints return raw arrays.

The frontend cannot reliably know:

- total rows
- total pages
- whether more data exists.

M-03 — Search is potentially expensive

Lead search uses multiple `ILIKE '%query%'` expressions.

At scale, this will become expensive without appropriate indexes/search strategy.

M-04 — Dedupe algorithm limits fuzzy scan to 300 leads

The data quality engine only examines a limited set of active leads.

This means it is not a complete organization-wide dedupe engine.

M-05 — Fuzzy dedupe pairing is window-limited

The comparison loop only checks a small number of neighboring records rather than all relevant candidate pairs.

This can miss duplicates depending on row ordering.

M-06 — Automatic stale lead archiving changes lifecycle state

The dedupe task changes:

`OPEN → STALE`

based on:

- 45 days
- zero recent activities
- | 4 calls.

This is a business lifecycle mutation happening inside a data-quality task.

It needs explicit ownership in the Lead Lifecycle domain.

M-07 — EOD reports calculate metrics independently from Telecaller KPI logic

There are separate calculations for:

- calls
- connects
- talk time
- conversions.

These should come from a centralized metrics service/query layer.

M-08 — Conversion is calculated differently across flows

Some paths use:

```
Lead.status == WON .
```

Other paths use:

```
pipeline stage is_won .
```

Payment logic modifies both.

These need one authoritative conversion definition.

M-09 — Call status values are free-form strings

Many statuses are represented as strings:

```
CONNECTED , INTERESTED , CALLBACK , INITIATED , COMPLETED , etc.
```

There is insufficient central validation.

M-10 — Activity status is overloaded

`Activity.status` is used for different concepts:

- communication status
- call outcome

- lifecycle status
- completion status.

These should be separated or formally enumerated.

M-11 — Error handling is inconsistent

Some endpoints return:

- 400
- 403
- 404
- 500

with inconsistent payload formats.

The frontend therefore often uses:

```
err.message
```

instead of a predictable API error contract.

M-12 — Frontend optimistic state handling is limited

Several operations refresh entire datasets after mutation rather than updating authoritative state through a central state layer.

This can create stale UI during concurrent activity.

31. LOW-SEVERITY / TECHNICAL-DEBT FINDINGS

L-01 — Large virtual environment included in project archive

The repository contains approximately 192 MB of `jarvis_crm` virtual-environment files.

This should not be part of the source repository/archive.

L-02 — Node modules are also included in the supplied project

`frontend/node_modules` exists in the archive.

This creates unnecessary repository size and environment drift.

L-03 — Built frontend `dist` is included

The repository contains an already-built `frontend/dist`.

Production builds should generally be generated by CI/CD.

L-04 — Multiple SQL schema artifacts exist

The repository contains:

- `jarvis_crm_neon_schema.sql`
- `final_proposed_db_Jarvs_CRM.sql`

These need explicit version ownership.

L-05 — Bundled SQLite database is stale

The included `jarvis_crm.db` contains only 25 tables.

The current SQLAlchemy model metadata contains 68 tables.

The bundled DB is therefore not representative of the current application schema.

L-06 — Project documentation and implementation have drifted

The Bible states:

- app-level RLS
- service-layer dominance

- global pull logs
- global contacts
- canonical database structure.

The implementation does not completely match those claims.

32. DATABASE → MODEL → SERVICE → API → FRONTEND SOURCE-OF-TRUTH MAP

Lead

Current flow

Database:

`Leads`

Model:

`Lead`

Service:

`lead_service.py`

API:

`leads.py`

Frontend:

`LeadsTable`, `LeadDetailModal`, `TelecallerDesk`, `KanbanBoard`

Problem

Lead state is split between:

- `lead.status`
- `pipeline_stage_id`
- `LeadStageHistory`

Ownership is split between:

- `Lead.owner_id`
- `lead_assignments`

Product is split between:

- `product_service_id`
- `product_service_name`

Required authoritative model

The future fix should establish:

Pipeline Stage = authoritative lifecycle state

Lead status = derived/controlled state

owner_id = current primary owner

lead_assignments = immutable assignment history/current assignment ledger

33. FOLLOW-UP SOURCE OF TRUTH

Current sources:

1. `tasks`
2. `execute_followup_policy()`
3. `create_task()`
4. frontend `TelecallerDesk`
5. Next-Best-Action
6. Daily Queue
7. Follow-up API.

This is too many places.

The authoritative source should eventually be:

`tasks` containing one active FOLLOW_UP per lead under the defined business rule.

Recommendations should never create another follow-up merely because they recommend one.

34. TARGET SOURCE OF TRUTH

Current sources:

1. `users.telecaller_targets`
2. `telecaller_targets`
3. `initialize_daily_targets()`
4. Team Management frontend
5. Telecaller Desk
6. EOD report.

This is currently one of the clearest architectural conflicts in the application.

There must be exactly one authoritative target record per:

organization + telecaller + business date

with explicit fields for:

- calls
- connected calls
- talk time
- qualified leads
- conversions
- revenue.

35. CALL SOURCE OF TRUTH

Current sources:

1. native dial action
2. `CallRecord`

3. initial CALL Activity
4. webhook CALL Activity
5. manually submitted CALL Activity
6. frontend duration.

The recommended future architecture should define:

CallRecord = authoritative telephony event

Activity = timeline projection

Activities should not independently become the source for raw call counting unless each Activity is guaranteed to represent exactly one real call.

36. GLOBAL REGISTRY SOURCE OF TRUTH

Current sources:

1. `global_companies`
2. `global_contacts`
3. `global_people`
4. tenant `companies`
5. tenant `contacts`
6. `source_global_company_id`
7. `source_global_contact_id`
8. historical `global_data_pull_logs`
9. pull status fields that were later removed
10. tenant-side existence checks.

This needs significant architectural cleanup.

The authoritative model should eventually be:

Global Registry Master Record

-

Global Claim/Pull Ledger

-

Tenant Projection

The tenant copy must never replace the global master.

37. IMPORT SOURCE OF TRUTH

Current flow:

Upload → physical file path → preview → client sends path → validation → ImportJob → Celery → tenant/public schema manipulation → lead creation.

The design is workable but the physical path should not be a client-controlled business identifier.

The authoritative object should be:

ImportJob ID

with server-controlled storage location.

38. TENANT ISOLATION ASSESSMENT

The project has a good conceptual foundation:

- authenticated user
- tenant resolution
- organization ID
- role checks
- soft deletion
- support sessions
- token versioning
- tenant schemas.

However, the implementation mixes:

row-level tenant filtering

with:

schema-per-tenant isolation

without fully standardizing which tables belong to which model.

This increases the attack surface.

The strongest concern is not merely missing `organization_id`.

The concern is:

Can every database query prove that the authenticated actor is allowed to access the specific object?

Currently, the answer is no.

39. AUTHENTICATION ASSESSMENT

The authentication foundation is comparatively strong.

Positive findings:

- JWT signature verification
- token expiration
- token versioning
- revoked-token support
- password hashing
- password-change session invalidation
- support-session token type
- organization suspension handling.

However, production hardening is still required because of:

- default secret
- DEBUG default
- exception leakage

- webhook authentication
 - permission override boundary
 - inconsistent role enforcement.
-

40. FRONTEND AUDIT

The frontend architecture is a single React application with configuration-driven navigation.

The Navbar is already partially designed around:

`NAVIGATION_CONFIG`

and includes a `More` mechanism.

This is a good direction.

However, there is still route-security duplication.

`Navbar.tsx` decides visibility.

`App.tsx` separately decides route authorization.

Backend APIs then perform another layer.

Therefore there are three authorization representations:

1. Navbar visibility
2. frontend route guard
3. backend authorization.

The backend must always remain authoritative.

41. FRONTEND PRODUCTS ISSUE

`ProductsServicesView.tsx` expects:

`status`

but the backend model uses:

`is_active`.

The delete button sends:

```
is_active: false.
```

Therefore the frontend/backend contract is inconsistent.

This is a genuine implementation bug rather than merely a design issue.

42. TELECALLER DESK ASSESSMENT

The Telecaller Desk has many intended features:

- daily queue
- next actions
- targets
- shift state
- calling
- WhatsApp
- email
- payment
- pre-call context
- outcome logging
- follow-up selection
- masked PII.

The UI is substantially developed.

However, the backend authority behind several of these features is not yet reliable.

The largest problems are:

1. Call duplication
2. Fake duration
3. Follow-up duplication/source conflict

4. Target source conflict
 5. same-tenant IDOR
 6. direct lead lifecycle modification
 7. UTC/business-day mismatch.
-

43. SHIFT / ATTENDANCE ASSESSMENT

The frontend correctly calls:

```
getShiftStatus()
```

and does not simply initialize the user as ACTIVE.

This is a positive design choice.

However, backend shift queries use user/date/session conditions and need:

- unique active-session enforcement
- organization consistency
- database-level protection against concurrent starts
- organization timezone handling.

The timer itself should remain a display derived from authoritative server timestamps.

44. TESTING ASSESSMENT

The test suite contains useful coverage for:

- login
- masking
- tenant isolation
- assignments
- global pull
- AI

- imports
- pipeline history
- platform RBAC
- support sessions
- token revocation
- audit chain.

However, the existing tests heavily favor positive scenarios.

Missing or insufficiently represented critical tests include:

- Telecaller accessing another Telecaller's lead by ID
- Telecaller modifying another Telecaller's lead
- Telecaller reading another user's activity
- ORG_ADMIN attempting platform permission escalation
- forged telephony webhook
- forged payment webhook
- repeated payment webhook
- concurrent global pull
- quota deduction with missing IDs
- quota deduction with already-pulled IDs
- cross-tenant target owner
- cross-tenant Product/Service assignment
- duplicate follow-up creation
- concurrent follow-up creation
- frontend-generated follow-up + policy-generated follow-up
- UTC vs organization timezone
- duplicate call Activities
- actual call duration

- direct status mutation bypassing stage history
- clean database migration from zero.

These tests are essential before production.

45. VERIFICATION RESULTS

Backend test suite

Attempted:

```
python -m pytest tests -q
```

Result:

NOT EXECUTED

Reason:

```
ModuleNotFoundError: No module named 'psycpg2'
```

This is an environment/dependency limitation.

No test result should be represented as passing.

Frontend production build

Attempted:

```
npm run build
```

Result:

FAILED BEFORE BUILD

Errors:

- Cannot find type definition file for `vite/client`
- Cannot find type definition file for `node`

The supplied environment's frontend dependencies are incomplete/inconsistent.

Clean Alembic migration

Attempted against a fresh SQLite database.

Result:

FAILED

Failure at:

```
3280ba601d18_catchup_missing_columns.py
```

due to unnamed foreign-key constraints in Alembic batch migration.

This is a confirmed migration compatibility failure.

46. IMPORTANT DISTINCTION: CONFIRMED BUG VS ARCHITECTURAL RISK

Confirmed through code

The following are directly visible implementation defects:

- duplicate update functions
- duplicated target sources
- hardcoded target defaults
- frontend/backend ProductService field mismatch
- direct Lead status mutation
- call Activity duplication
- hardcoded call durations
- quota deduction using requested count
- missing webhook authentication/signatures
- permission override exposure
- missing Telecaller ownership on several object-level endpoints
- stale bundled database

- clean SQLite migration failure
- contradictory migration operations
- global exception detail leakage
- default production secret
- UTC scheduling mismatch.

Architectural risks requiring production verification

These require actual PostgreSQL environment verification before being declared runtime failures:

- exact tenant-schema FK behavior
- connection-pool search_path leakage under concurrent traffic
- PostgreSQL migration behavior at current head
- exact cross-tenant behavior of every endpoint
- external provider webhook behavior
- Celery retry duplication under real broker failure.

These should remain classified as risks until reproduced.

47. PRIORITY ORDER FOR THE FUTURE FIX PHASE

The fixing phase should NOT start with UI polish.

Recommended order:

Phase 0 — Source-of-Truth Lock

Define authoritative ownership for:

- Lead lifecycle
- Lead ownership
- Call

- Activity
- Follow-up
- Target
- Conversion
- Global Claim
- Product/Service
- Attendance.

No feature should be modified before these definitions are agreed.

Phase 1 — Security & Tenant Boundary

Fix:

- Telecaller IDOR
 - permission escalation
 - webhook authentication
 - payment authorization
 - Global Registry role access
 - target owner validation
 - tenant schema uniqueness
 - default secret
 - exception leakage
 - CORS
 - import path trust.
-

Phase 2 — Database & Migration Canonicalization

Establish one authoritative database strategy.

Choose explicitly:

shared schema with organization_id

OR

schema-per-tenant

OR a rigorously defined hybrid where every table category is documented.

Then:

- repair Alembic chain
 - remove destructive contradictions
 - eliminate startup `create_all`
 - eliminate ad-hoc schema migrations
 - define migration ownership
 - add required constraints/indexes
 - establish clean DB creation test
 - establish upgrade-existing-DB test.
-

Phase 3 — Lead Lifecycle

Make stage/lifecycle authoritative.

Enforce:

- stage transition service
 - conversion rules
 - assignment rules
 - status derivation
 - immutable stage history
 - ownership consistency.
-

Phase 4 — Call & Activity Architecture

Define:

CallRecord = actual call

Activity = timeline event

Then ensure:

one real call = one qualifying call event.

Remove fabricated duration and duplicated call counting.

Phase 5 — Follow-up Engine

Define:

Task = authoritative active follow-up record

Then make:

- policy engine
- reschedule
- completion
- queue
- Next-Best-Action

all operate against that single record.

Recommendations must not create duplicate tasks.

Phase 6 — Target Engine

Remove:

`users.telecaller_targets`

as an independent source.

Use:

`telecaller_targets`

with:

organization + telecaller + business date

as the authoritative target identity.

Phase 7 — Global Registry

Create a proper claim ledger.

The correct conceptual flow should be:

Global Master

↓

Atomic Claim

↓

Claim Ledger

↓

Tenant Projection

↓

Lineage

Quota should be deducted only after successful claim.

Phase 8 — Import System

Use:

Upload ID → Import Job → Server-owned file → Validation → Execution
rather than allowing client-supplied filesystem paths.

Phase 9 — Analytics

Centralize metrics.

Dashboard, Telecaller Desk, EOD and reports should all consume the same metric definitions.

Phase 10 — Frontend Contract Cleanup

Synchronize:

- TypeScript interfaces

- Pydantic schemas
 - API responses
 - loading/error states
 - Product/Service fields
 - routes
 - navigation
 - pagination.
-

48. FINAL AUDIT VERDICT

JARVIS CRM is **not a superficial prototype**. There is substantial architecture and functionality already implemented.

The project has particularly good foundations around:

- role-aware UI
- JWT authentication
- token versioning
- support sessions
- audit concepts
- masking
- tenant context
- background processing
- pipeline history
- imports
- global intelligence
- Telecaller workflow concepts.

The primary problem is that the application has grown through multiple implementation phases and now contains **architectural drift**.

The biggest issue is not "missing features."

It is:

Multiple implementations of the same business truth.

The most dangerous examples are:

Targets:

`users.telecaller_targets` VS `telecaller_targets`

Calls:

`CallRecord` vs multiple CALL Activities

Follow-ups:

policy engine vs frontend-created tasks

Lead lifecycle:

stage vs status

Ownership:

`owner_id` VS `lead_assignments`

Global Registry:

master record vs tenant projection vs historical pull-log concept

Database:

Alembic vs `create_all()` vs startup SQL vs tenant-schema cloning

Authorization:

frontend visibility vs route guard vs endpoint authorization.

Until these conflicts are resolved, fixing individual UI symptoms will repeatedly create new inconsistencies.

49. FINAL CLASSIFICATION

CRITICAL

The current audit identified **multiple Critical findings**, primarily involving:

- tenant/object authorization
- privilege escalation
- webhook trust
- payment lifecycle manipulation
- Global Registry claiming/quota
- migration integrity
- database source-of-truth
- lead lifecycle integrity
- Telecaller metric correctness
- target source-of-truth
- timezone correctness.

HIGH

High-severity findings primarily involve:

- follow-up architecture
- assignment consistency
- tenant-schema architecture
- import security
- communication integrations
- Products & Services
- payment idempotency
- global-data architecture
- service/API duplication.

MEDIUM

Medium findings primarily involve:

- analytics consistency

- pagination
- dedupe scalability
- status enums
- error contracts
- stale frontend state
- search performance.

LOW

Low findings are mostly:

- repository hygiene
- bundled environments
- generated builds
- duplicate schema artifacts
- documentation drift.

50. MOST IMPORTANT CONCLUSION

The next phase should **not** be:

┆ "Fix all the bugs one by one."

It should be:

┆ **"Use this audit to establish the canonical architecture and sources of truth, then fix each subsystem against those definitions."**

The correct implementation order is:

Source of Truth → Security/Tenant Boundary → Database/Migrations → Lead Lifecycle → Calls → Follow-ups → Targets → Global Registry → Imports → Analytics → Frontend → Integrations → Full Regression Testing

Only after those layers are corrected should the final production-readiness assessment be performed.

No fixes were applied as part of this audit. This report represents the analysis/baseline that should be used for the subsequent fixing and debugging phase.